

WORKSHOP N°2

DATE: 07-02-25

COMPILERS

PROFESSOR: Eng. Carlos Andrés Sierra Virgüez, M.Sc.

STUDENT: Luis Alejandro Cely Díez - 20211020093

INSTITUTION: Francisco José de Caldas District University

① For each one of next cases define a regular expression as used in a compiler based on the Python re library.

a) Identifier: A regular expression to match valid identifiers (Variable name, function names, etc.).

```
IDENTIFIER = re.compile(r'\b[a-zA-Z_][a-zA-Z0-9_]*\b')
```

b) Integer Literal: A regular expression to match integer literals

```
INTEGER = re.compile(r'\b[-]?[0-9]+\b')
```

c) Floating Point Literal: A regular expression to match floating point literals

```
FLOAT = re.compile(r'\b[-]?[0-9]+\.[0-9]+\b')
```

d) String Literal: A regular expression to match strings literals enclosed in double quotes

```
STRING = re.compile(r'\b"[^"]*"'\b')
```

e) Single Line Comments: A regular expression to match single line comments starting with '//'

```
COMMENT = re.compile(r'\b//[^\n]*\b')
```

f) Multi-line Comments: A regular expression to match multi-line comments enclosed in '/* */'

```
MULTICOMMENT = re.compile(r'\b/*[^\n]*\b')
```

g) Whitespace: A regular expression to match whitespace characters (spaces, tabs, newlines)

```
WHITESPACE = re.compile(r'\s+')
```

h) Operators: A regular expression to match common operators
(e.g., '+', '-', '*', '/', '=', '!=')

OPERATOR = re.compile(r'[+=*/<>!\%\'-]|==|!=|<=|>=')

i) Keywords: A regular expression to match reserved keywords
(e.g., 'if', 'else', 'while', 'return')

KEYWORD = re.compile(r'\b(if|else|while|for|return|void|class') etc...

j) Hexadecimal Literal: A regular expression to match hexadecimal literals

HEXADECIMAL = re.compile(r'\b0[xX][0-9a-fA-F]+\b')

2) Be G a context free grammar and based on a provided productions, create derivation trees for the following statements

a) $x = 5 + 3 * 2;$

Become $\langle \lambda \rangle$

$S \rightarrow \text{Program}$

$\rightarrow \text{StatementList}$

$\rightarrow \text{Statement StatementList}$

$\rightarrow \text{Assignment}$

$\rightarrow \text{Identifier "=" Expression ";"}$

$\rightarrow x = \text{Term Expression ";"}$

$\rightarrow x = \text{Factor Term "+" Term Expression}$

$\rightarrow x = 5 "+" \text{Factor Term}$

$\rightarrow x = 5 "+" 3 "*" \text{Factor Term}$

$\rightarrow x = 5 "+" 3 "*" 2$

b) $\text{if}(x > 0) \{$

$y = x - 1;$

$\} \text{else} \{$

$y = 0$

$\}$

Become $\langle \text{lambda} \rangle$

$S \rightarrow \text{Program}$

$\rightarrow \text{StatementList}$

$\rightarrow \text{statement StatementList}$

$\rightarrow \text{IfStatement}$

$\rightarrow \text{"if" "(" Expression ")" "{" StatementList "}" ElsePart}$

$\rightarrow \text{"if" "(" Term Expression ")" "{" StatementList "}" ElsePart}$

$\rightarrow \text{if (Factor Term') \{ Assignment \} else \{ StatementList \}}$

$\rightarrow \text{if (Identifier > Factor Term') \{ Identifier = Expression; \} else \{ StatementList \}}$

$\rightarrow \text{if (x > Number) \{ y = Term Expression; \} else \{ statement StatementList \}}$

$\rightarrow \text{if (x > 0) \{ y = Factor Term' - Term Expression; \} else \{ Assignment \}}$

$\rightarrow \text{if (x > 0) \{ y = Identifier - Factor Term'; \} else \{ Identifier = Expression; \}}$

$\rightarrow \text{if (x > 0) \{ y = x - Number; \} else \{ y = Term Expression; \}}$

$\rightarrow \text{if (x > 0) \{ y = x - 1; \} else \{ y = Factor Term'; \}}$

$\rightarrow \text{if (x > 0) \{ y = x - 1; \} else \{ y = Number; \}}$

$\rightarrow \text{if (x > 0) \{ y = x - 1; \} else \{ y = 0; \}}$

c) while (x < 10) {
 x = x + 1;
}

Become **<lambda>**

S → Program

→ StatementList

→ Statement StatementList

→ WhileStatement

→ while (Expression) { statementList }

→ while (Term Expression') { Statement StatementList }

→ while (Factor Term' < Term Expression') { Assignment }

→ while (Identifier < Factor Term') { Identifier = Expression ; }

→ while (x < Number) { x = Term Expression' ; }

→ while (x < 10) { x = Factor Term' + Term Expression' ; }

→ while (x < 10) { x = Identifier + Factor Term' ; }

→ while (x < 10) { x = x + Number ; }

→ while (x < 10) { x = x + 1 ; }

d) return (a+b) * C ;

Become **<lambda>**

S → Program

→ StatementList

→ Statement StatementList

→ ReturnStatement

→ return Expression ;

→ return Term Expression' ;

→ return Factor Term' ;

→ return (Expression) * Factor Term' ;

→ return (Term Expression') * Identifier ;

→ return (Factor Term' + Term Expression') * C ;

→ return (Identifier + Factor Term') * C ;

→ return (a + Identifier) * C ;

→ return (a + b) * C ;